

CS201 – Data Structures II

Lecture 04

Dr. Muhammad Mobeen Movania

Outline

- Complexity Analysis
- Asymptotic Analysis
- Asymptotic notation
- Amortized Analysis
 - Aggregate analysis
 - Accounting method

What have we covered so far?

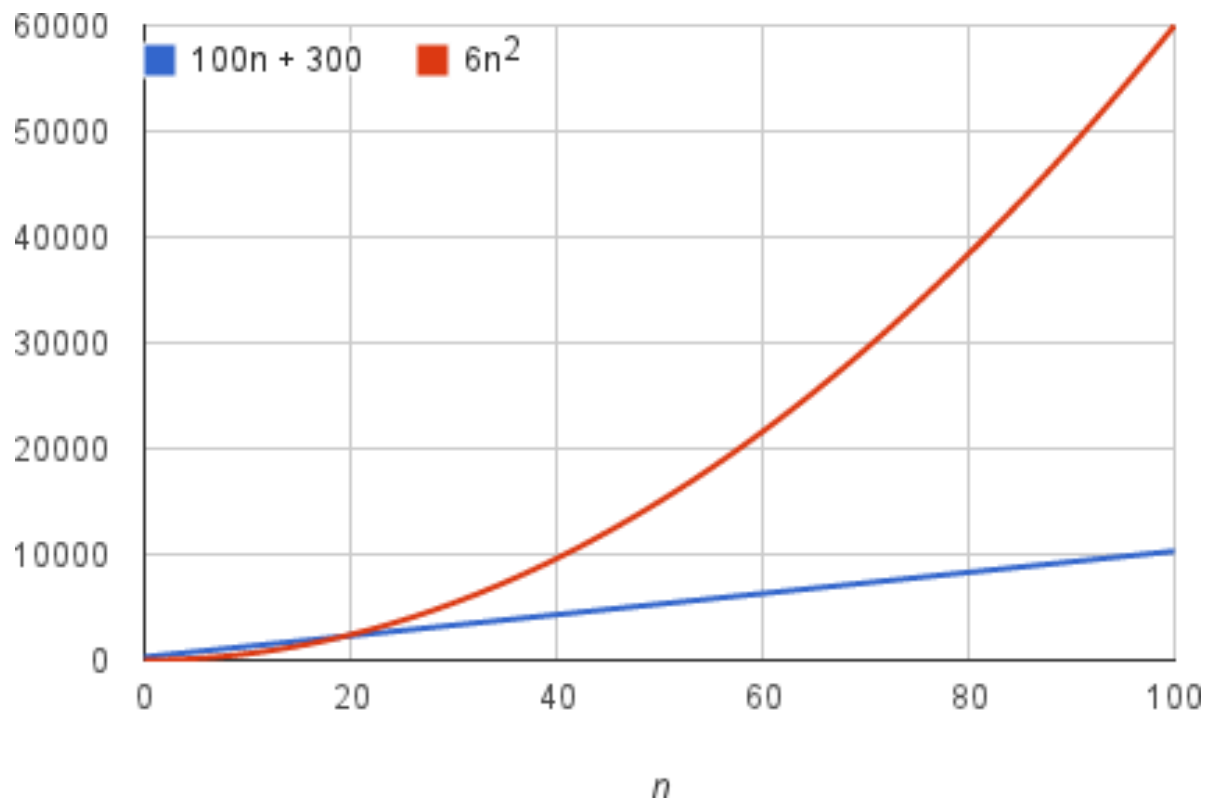
- ArrayStack
 - Implements the List interface
- ArrayQueue
 - Implements the Queue interface (FIFO discipline)
- ArrayDeque
 - Implements the Deque interface
- We will now move to complexity analysis

Complexity analysis

- The running time of an algorithm depends on how long it takes a computer to run the algorithm.
- This varies due to several factors
 - System configuration(Type/Amount of RAM, Processor speed, etc.)
 - Programming language and compiler used
- Instead of focusing on running time, we focus on two things:
 - how long the algorithm takes in terms of **input size**
 - how fast a function grows with the **input size**

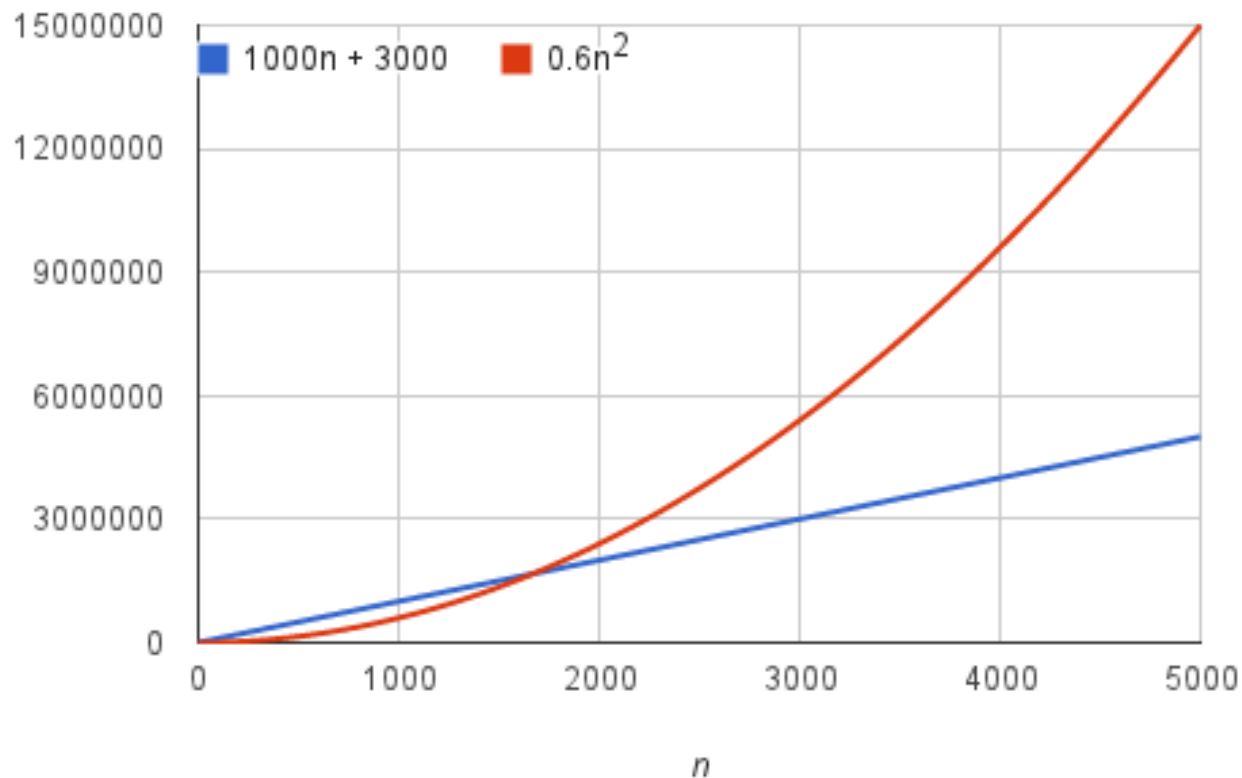
Asymptotic analysis

- Suppose an algorithm takes
 - $6n^2 + 100n + 300$



Asymptotic analysis

- Suppose an algorithm takes
 - $0.6n^2 + 1000n + 3000$



Asymptotic notation

- In asymptotic notation
 - we just take n term with the highest power and ignore the constant and lower order terms of n
- Why?
 - For a large value of n, the highest power term of n dominates
- Example
 - $6n^2 + 100n + 300 \rightarrow$ running time is n^2
 - $0.6n^2 + 1000n + 3000 \rightarrow$ running time is n^2

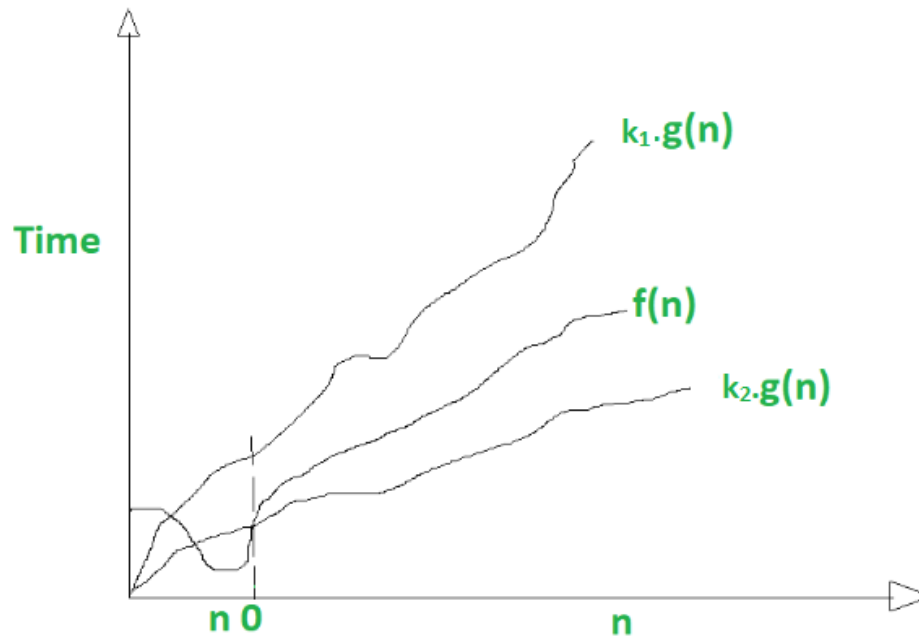
Asymptotic notations

- Three forms
 - big- O (asymptotic upper bound)
 - big- Ω (asymptotic lower bound)
 - big- Θ (asymptotic tight bound)
- Big- O
 - tells which functions grow at a rate $\geq f(n)$, for large n
- Big- Ω
 - tells which functions grow at a rate $\leq f(n)$, for large n
- Big- Θ
 - tells which functions grow at the same rate as $f(n)$, for large n

Big- Θ

- Once n gets large enough, the running time is at least $k_1 \cdot n$ and at most $k_2 \cdot n$ for some constants k_1 and k_2

$$0 \leq k_2 g(n) \leq f(n) \leq k_1 g(n) \text{ for all } n \geq n_0$$



Example

- Consider the following code

```
var doLinearSearch = function(array, targetValue)
{
    
    for (var guess = 0; guess < array.length; guess++)
    {
        if (array[guess] === targetValue)
        {
            return guess;
            // found it!
        }
    }
    return -1; // didn't find it
};
```

 **c1**

 **n**

 **c2**

- Running time = $c_1 \cdot n + c_2$

Example

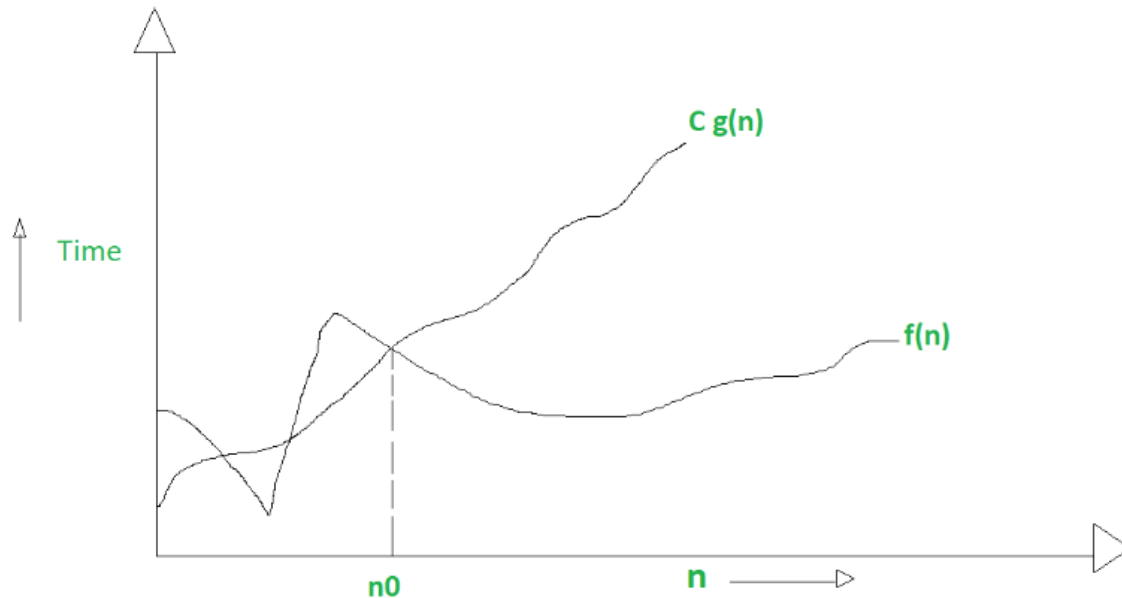
- The constants c_1 and c_2 don't tell us anything about the rate of growth of the running time but the running time grows with the array size
- The worst-case running time of the given code **grows like the array size n**
- Running time is $\Theta(n)$

Discussion

- If we want to know that the running time grows **at most x** but it could grow more slowly
- This is where Big-O notation comes in
- Example:
 - Binary search has a worst-case complexity $\Theta(\log_2 n)$ that is it cannot have a running time growth $> \log_2 n$
 - But we cannot say that the running time is $\Theta(\log_2 n)$
 - this is not always the case, as depending on the given value may be the item is found in the first iteration that is running time is $\Theta(1)$
 - We can say that the running time of binary search is always $O(\log_2 n)$ as it captures all runtime growths up to $\log_2 n$

Big O Notation

- Let $f(n)$ define running time of an algorithm.
- We say that $f(n)$ is $O(g(n))$ if and only if
$$0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0$$

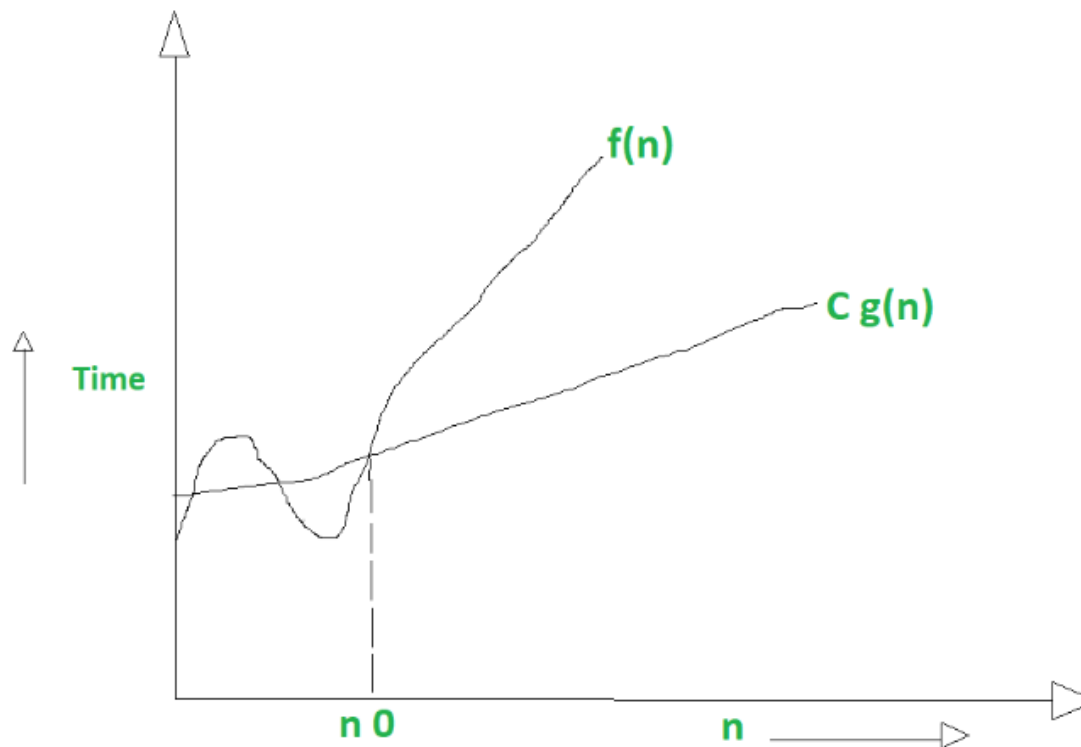


Discussion

- Sometimes, we want to say that the running time of an algorithm grows **at least x** but it could grow more rapidly
- This is where Big- Ω comes in

Big Omega Notation (Ω)

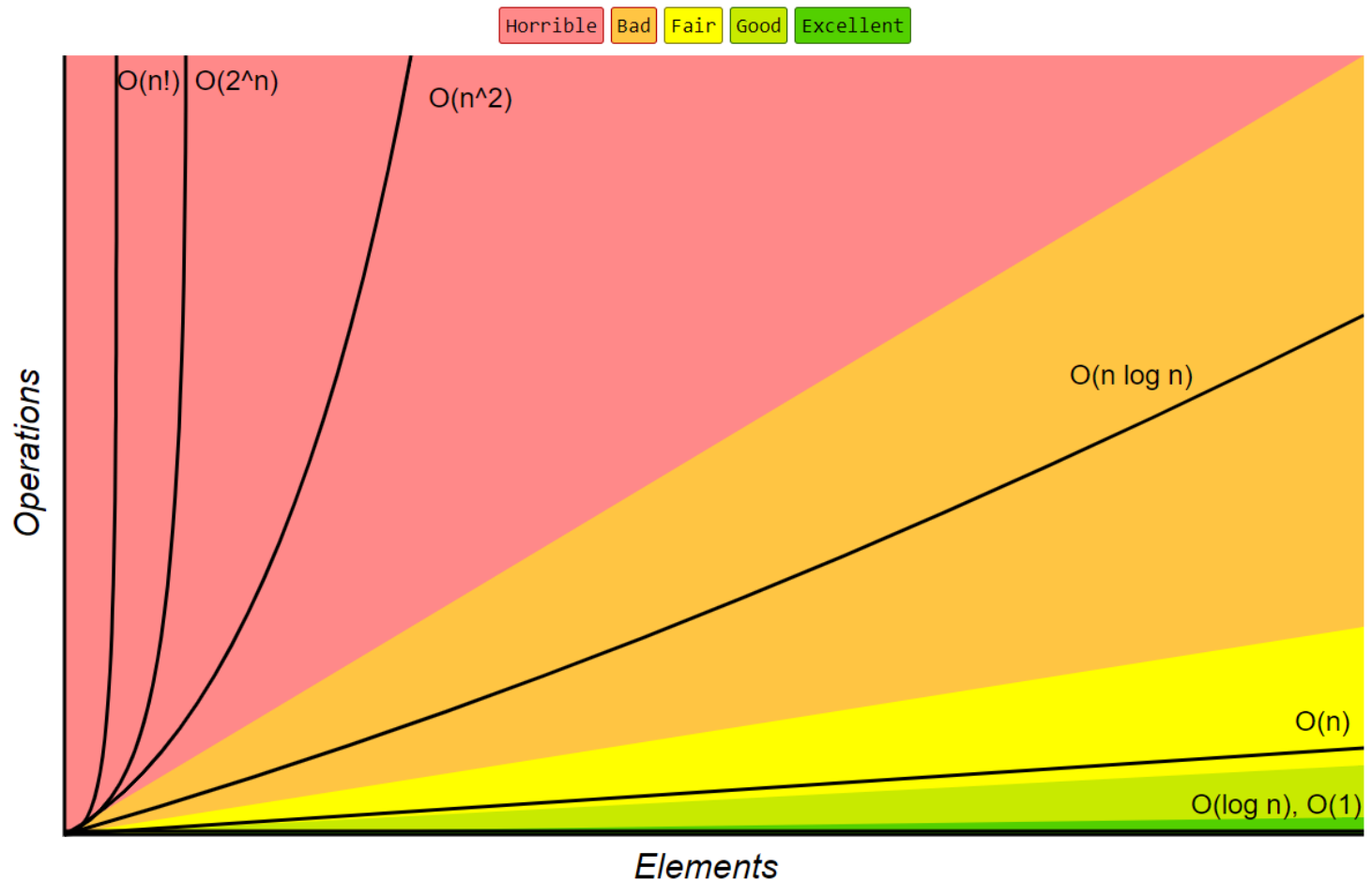
- $f(n)$ is said to be $\Omega(g(n))$ if,
 $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0$



Functions and their complexities

1. $\Theta(1)$
 2. $\Theta(\log_2 n)$
 3. $\Theta(n)$
 4. $\Theta(n \log_2 n)$
 5. $\Theta(n^2)$
 6. $\Theta(n^2 \log_2 n)$
 7. $\Theta(n^3)$
 8. $\Theta(2^n)$
 9. $\Theta(n!)$
- List of functions ordered by slowest to fastest growing
 - The fastest the growth, the worst the performance
 - Note that an exponential function a^n ($a > 1$) grows faster than any polynomial function n^b (b is any constant)

Big O complexity chart



Amortized Analysis

- It is used for algorithms where an occasional operation is very slow, but most of the other operations are faster
- We will use amortized analysis for calculating the runtime cost of the ArrayStack resize function

Amortized Analysis – ArrayStack resize

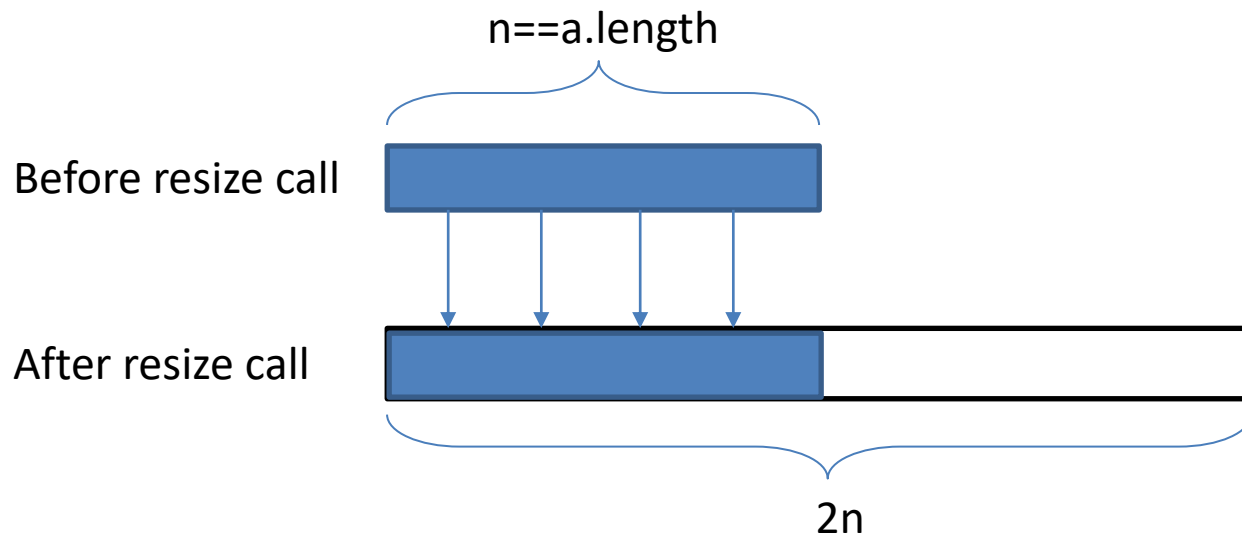
- Get and set functions have $O(1)$ complexity
- Ignoring the cost of resize, add and remove each run in $O(1+n-i)$ time
- We calculate the cost of resize function using amortized analysis
- We will prove that resize calls do not add much to the overall time

Amortized analysis - resize

Lemma 2.1. *If an empty ArrayStack is created and any sequence of $m \geq 1$ calls to `add(i, x)` and `remove(i)` are performed, then the total time spent during all calls to `resize()` is $O(m)$.*

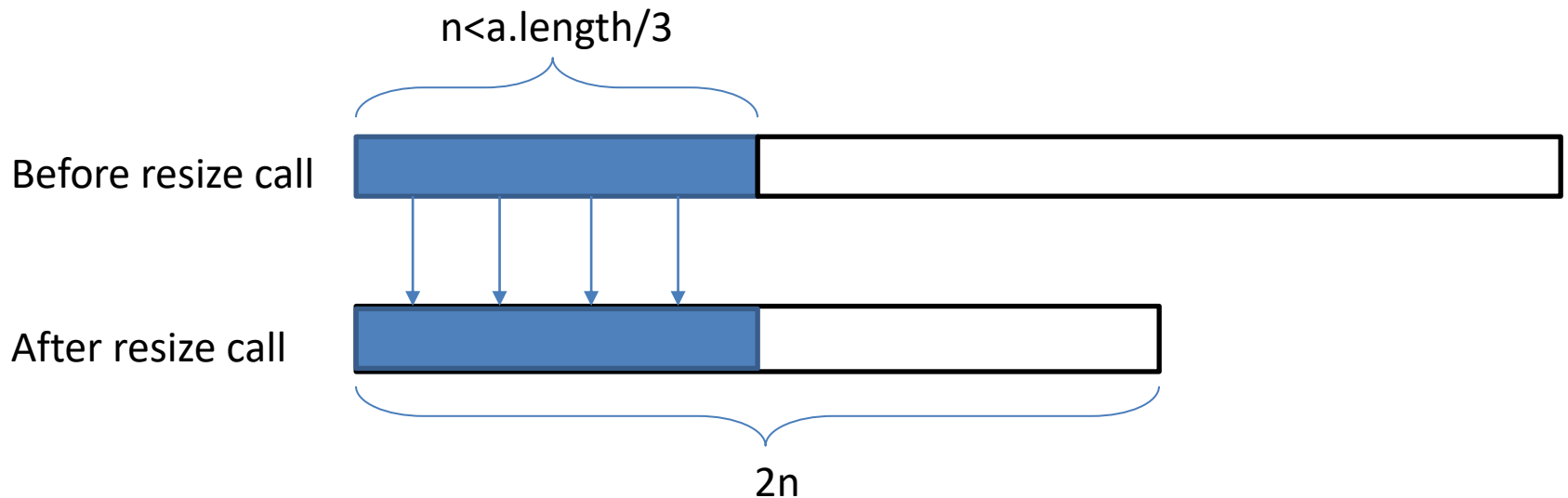
Amortized analysis - resize

- resize during `add(i,x)`



Amortized analysis - resize

- resize during remove(i)

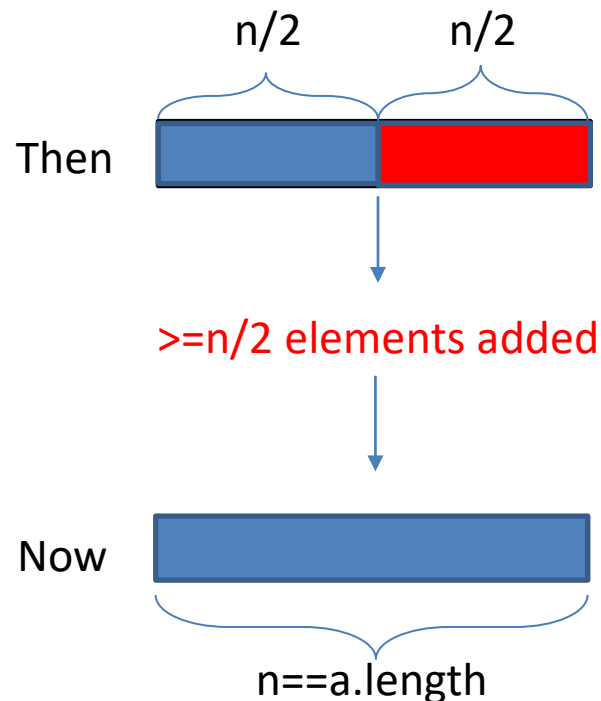


Amortized analysis - resize

- We will study the number of elements copied during all calls to `resize`
- Why?
 - All the work done in `resize` is proportional to the number of elements copied
- Plan
 - Show that the total number of elements copied by `resize()` is at most $2m$

Amortized analysis - resize

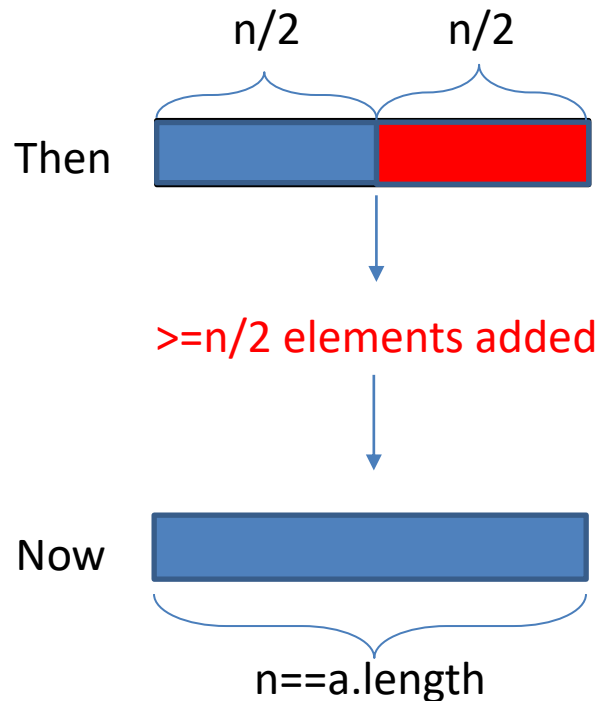
- Lets look at the add call before triggering the resize



- So an additional $n/2$ elements were added since the last resize call

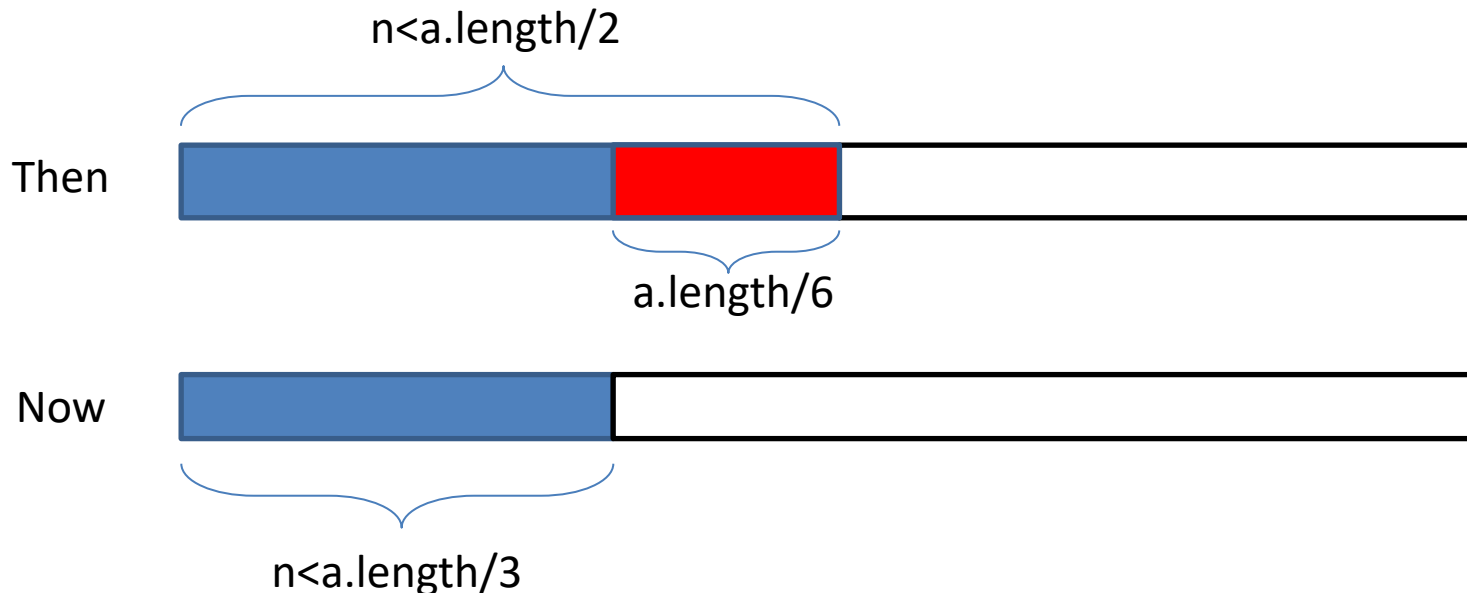
Amortized analysis - resize

- So between previous and current resize operation, at least $n/2$ elements were added



Amortized analysis - resize

- Lets look at the remove call before triggering the resize



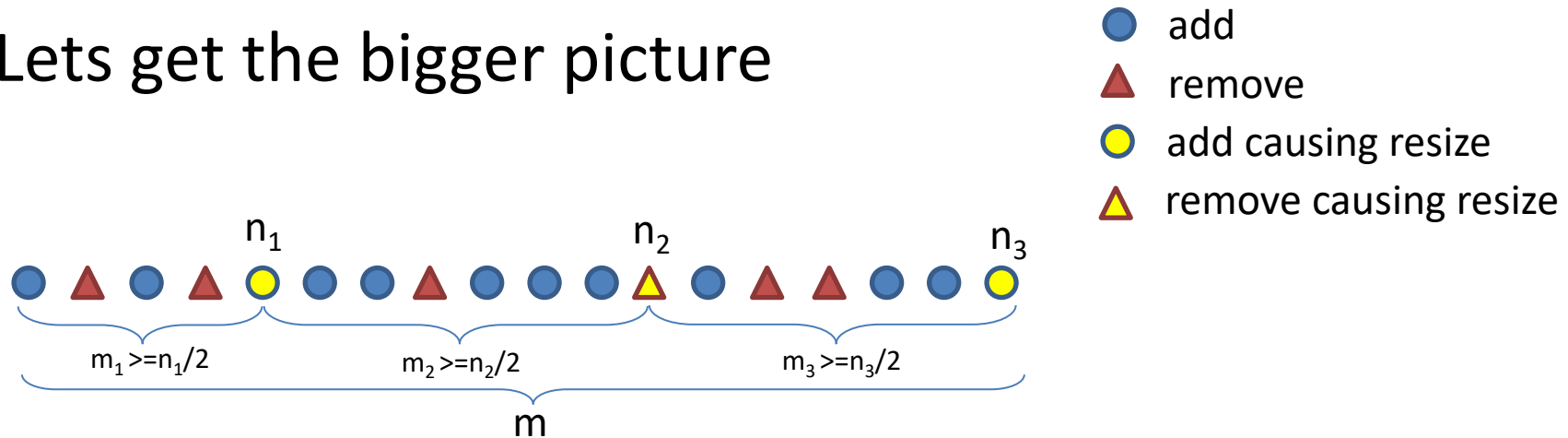
- At least $n/2$ remove calls were made between resize calls

Amortized analysis - resize

- Crux so far
 - If a resize operation copies n elements, then there have been at least $n/2$ add or remove operations since the preceding resize operation

Amortized analysis - resize

- Lets get the bigger picture



- n_i – number of items copied by i^{th} resize
- m_i - number of add/remove operations between $(i-1)^{\text{th}}$ and i^{th} resize
- Total number of items copied $N = \sum_{i=1}^R n_i$
- Total number of operations $m \geq \sum_{i=1}^{\infty} m_i$

Amortized analysis - resize

- Total number of items copied (N)
 - $N = n_1 + n_2 + n_3 + \dots$
- Total number of add/remove operations (m)
 - $m \geq m_1 + m_2 + m_3 + \dots$
 - $m \geq n_1/2 + n_2/2 + n_3/2 + \dots$
 - $m \geq N/2$
 - $N \leq 2m$
- Total number of items (N) copied by resize is at most $2m$, the total time spent in all calls to resize is $O(m)$

Amortized Analysis – Aggregate Analysis

- Consider a sequence of n operations
- Compute the worst case running time ($T(n)$) for a sequence of n operations

Amortized cost of one operation = $T(n)/n$

Example – Dynamic Array

- Plan
 - Initial size of array is 1
 - We add the new item into the array
 - Whenever the array is full, we double the current size, copy the existing elements and then append x into the array

Example – Dynamic array

add(1)

1

add(2)*

1	2
---	---

add(3)*

1	2	3	
---	---	---	--

add(4)

1	2	3	4
---	---	---	---

add(5)*

1	2	3	4	5			
---	---	---	---	---	--	--	--

add(6)

1	2	3	4	5	6		
---	---	---	---	---	---	--	--

add(7)

1	2	3	4	5	6	7	
---	---	---	---	---	---	---	--

add(8)

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Asymptotic complexity - add

- Worst case complexity of
 - single add operation: $O(n)$
 - n add operations: $n \cdot O(n) = O(n^2)$ (loose upper bound)

Example – Dynamic Array

- We determine the cost of each add operation

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
n_i	1	2	4	4	8	8	8	8	16	16	16	16	16	16	16	16
cost_i	1	2	3	1	5	1	1	1	9	1	1	1	1	1	1	1

- Cost of i^{th} add (C_i) = $\begin{cases} i, & \text{when } (i-1) \text{ is a power of } 2 \\ 1, & \text{otherwise} \end{cases}$

- Cost of n add operations = $n + \sum_{j=0}^{\log_2(n-1)} 2^j$

Example – Dynamic Array

$$\sum_{j=0}^{\log_2(n-1)} 2^j = 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{\log_2(n-1)}$$

$= a_0(r^m - 1)/(r - 1)$ (sum of geometric series formula)

First term: $a_0 = 2^0 = 1$

Ratio between terms: $r = 2$

Total terms: $m = \log_2(n-1) + 1$

$$\begin{aligned} &= (2^{\log_2(n-1)+1} - 1)/(2 - 1) \\ &= (2 \cdot 2^{\log_2(n-1)} - 1) = (2(n-1) - 1) = 2n - 2 - 1 = 2n - 3 \end{aligned}$$

$$\sum_{j=0}^{\log_2(n-1)} 2^j = 2n - 3$$

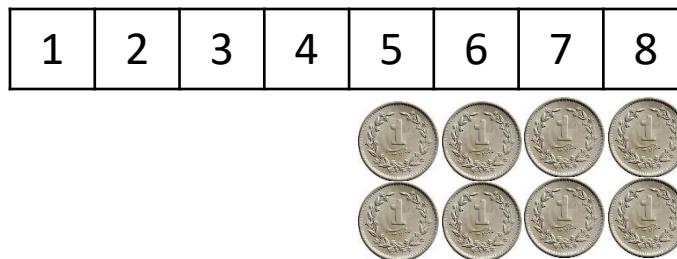
- Amortized cost of n add operations $T(n) = n + 2n - 3 = 3n - 3 < 3n = O(n)$
- Amortized cost of 1 add operation $= T(n)/n = 3n/n = 3 = O(1)$

Accounting Method

- We assign to each operation an amortized cost in terms of coins
- D_i is the data structure after i^{th} operation
- c_i is the cost of i^{th} operation
- $\hat{c}_i$ is the amortized cost of i^{th} operation = assigned coins
- Rules
 - If $\hat{c}_i > c_i$, save unused coins in D_i
 - If $\hat{c}_i < c_i$, pay the cost of operation using saved coins
 - Initial credit in $D_0 = 0$
- Cost invariant: saved coins ≥ 0
 - For all j : $\sum_{i=1}^j \hat{c}_i - \sum_{i=1}^j c_i \geq 0$
 - This implies that $\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$ for n operations

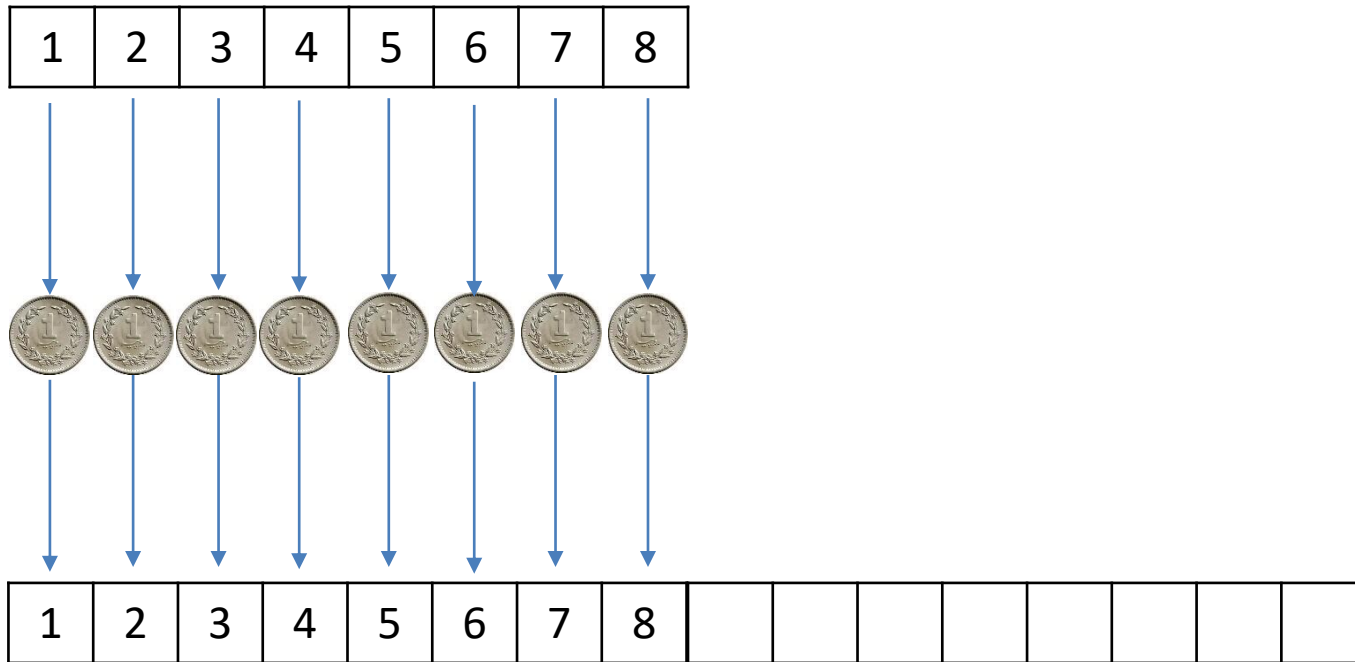
Example - Dynamic Array

- Actual cost $c_i = \begin{cases} 1 \text{ coin for 1 insertion} \\ n \text{ coins to copy } n \text{ elements to a new array} \end{cases}$
- Invariant
 - Every element in the right half has 2 coins
- Amortized cost $\hat{c}_i = 3$,
 - 1 coin to pay the actual cost of insertion and 2 coins to save



Example – Dynamic Array

- If array is full, the right half can pay the cost of copying



Example – Dynamic Array

- The worst case running time $T(n)$ of n appends operations is $O(n)$
- The amortized running time of 1 append operation is $O(\hat{c}_i) = O(1)$
- Invariant tells that the savings are always >0
- Total amortized cost of n operations $T(n)$
 - $\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i \leq \sum_{i=1}^n 3 \leq 3n = O(n)$
- Amortized cost of 1 operation:
 - $T(n)/n = 3n/n = 3 = O(1)$

Book Readings

- ODS: Chapter 2 pages: 29-43

Next Lecture

- Lecture 05 – Linked Lists